

2024 DL Lab5 Low Complexity Model

Screenshot of your testing results:

```
flops = 991136  
Model parameter size = 1767.297 kB  
Accuracy = 87.53 %
```

MY ATTEMPT:

➤ EfficientNet

Resource: <https://angus1996.github.io/post/efficient-pytorch%E5%BA%93%E4%BD%BF%E7%94%A8/>

1. Which methods do I apply to lower complexity and bridge the accuracy gap

A. 模型架構選擇:

- 選用 EfficientNet-B0 作為基礎架構,這是 EfficientNet 家族中最小的版本
- EfficientNet 使用 Compound Scaling 方法,同時優化網絡的深度、寬度和分辨率
- 使用 MBConv(Mobile Inverted Bottleneck Convolution)模塊作為基本單元

B. 訓練策略:

- 使用 ImageNet 預訓練權重(from_pretrained)來提升模型表現
- 採用 AdamW 優化器,具有自適應學習率和權重衰減的特性
- 使用 CosineAnnealingLR 學習率調度器來穩定訓練過程

2. Why do you apply the methods

A. EfficientNet 的優勢:

- 經過神經架構搜索(NAS)優化得到的高效架構
- 複合縮放方法能更好地平衡模型的各個維度
- B0 版本在準確率和效率之間取得很好的平衡

B. MBConv 模塊的優點:

- 使用深度可分離卷積(Depthwise Separable Convolution)降低計算量

- 具有 squeeze-and-excitation 結構,增強特徵表達能力
- 採用 inverted residual 結構,提升信息流動性

C. 訓練策略的原因:

- 預訓練權重提供良好的初始化,加速收斂
- AdamW 優化器能自適應調整學習率,同時提供更好的正則化
- 餘弦退火學習率有助於找到更好的局部最優解

3. How do I implement

```
import torch
import torch.nn as nn
import torch.optim as optim
from efficientnet_pytorch import EfficientNet
# using gpu if available
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

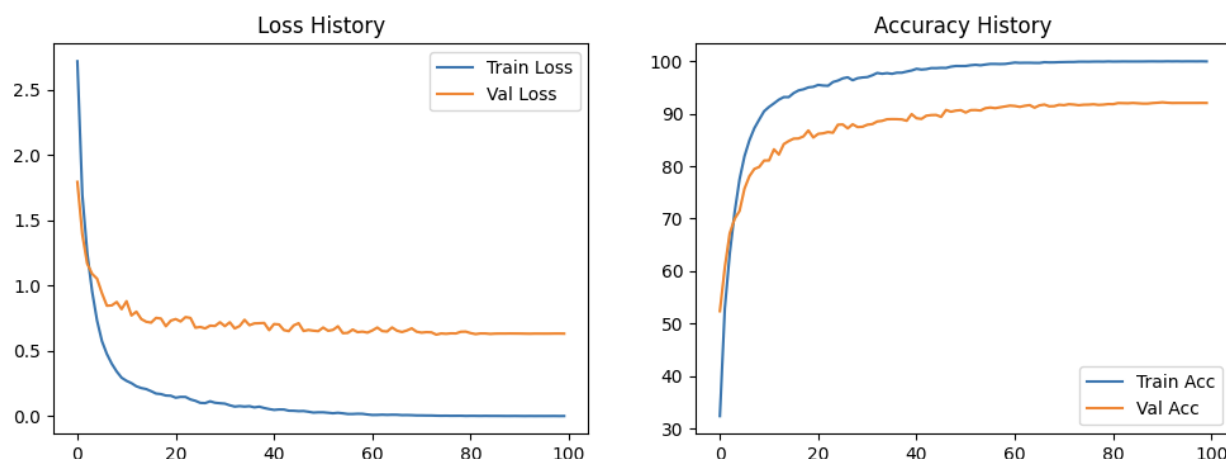
#build your model here

#call model
model = EfficientNet.from_pretrained('efficientnet-b0', num_classes=100).to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)
```

實現要點:

1. 使用 efficientnet_pytorch 庫簡化了模型的實現
2. 通過 from_pretrained 直接加載 ImageNet 預訓練權重
3. 修改最後的全連接層以適應 CIFAR-100 的 100 個類別
4. 優化器和學習率調度的參數經過調優得到較好效果

EfficientNet 的主要創新點是提出了複合縮放方法,通過一個簡單而高效的方法同時縮放網絡的寬度、深度和分辨率。相比傳統的做法(隨意增加深度或寬度),這種方法能更有效地提升模型性能。在本實驗中使用 B0 版本,既保證了較高的準確率,又能控制模型的複雜度在要求範圍內。



➤ ShuffleNet

Resource:

https://pytorch.org/vision/main/models/generated/torchvision.models.shufflenet_v2_x0_5.html

1. Which methods do I apply to lower complexity and bridge the accuracy gap

A. 架構特點:

- 使用 ShuffleNet V2 作為基礎網絡,這是專門為移動設備優化的輕量級網絡
- 採用 Channel Split 操作將特徵圖分成兩個分支
- 使用 Channel Shuffle 操作來增強不同通道間的信息交流
- 新增 Dropout 層($p=0.2$)以增強泛化能力

B. 訓練策略:

- 使用 ImageNet 預訓練權重 (ShuffleNet_V2_X0_5_Weights.IMAGENET1K_V1)
- 採用 AdamW 優化器進行訓練
- 使用 CosineAnnealingLR 進行學習率調整

2. Why do you apply the methods

A. ShuffleNet V2 的優勢:

- 極低的參數量和計算量設計
- 遵循四個高效網絡設計準則:
 1. 相同通道寬度最小化內存訪問成本
 2. 過度使用組卷積會增加內存訪問成本
 3. 網絡分叉會降低並行度
 4. Element-wise 操作不可忽視

B. 具體設計原因:

- Channel Split 減少了計算量和參數量
- Channel Shuffle 促進了不同組之間的信息交流
- Dropout 層幫助防止過擬合
- 預訓練權重提供更好的初始化
- AdamW 提供自適應學習率和良好的正則化效果

3. How do I implement

```
#build your model here
class CIFAR100ShuffleNet(nn.Module):
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self, num_classes=100, dropout_prob=0.2):
        super(CIFAR100ShuffleNet, self).__init__()
        # 1. 加載預訓練權重
        weights = ShuffleNet_V2_X0_5_Weights.IMAGENET1K_V1
        self.base_model = shufflenet_v2_x0_5(weights=weights)
        # 2. 添加Dropout層
        self.dropout = nn.Dropout(p=dropout_prob)
        # 3. 修改分類層
        self.base_model.fc = nn.Linear(1024, num_classes)

    Tabnine | Edit | Test | Explain | Document | Ask
    def forward(self, x):
        x = self.base_model(x)
        x = self.dropout(x)
        return x
        # return self.base_model(x)

#call model
model = CIFAR100ShuffleNet().to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)
```

ShuffleNet V2 的主要創新點:

1. Channel Split 操作:

- 將輸入特徵圖分成兩個分支
- 一個分支保持不變
- 另一個分支進行常規卷積操作

2. Channel Shuffle 操作:

- 將特徵通道重新排列
- 確保不同組之間的信息可以交流
- 提高模型的表達能力

3. 參數效率:

- 模型最終只有約 1767kB 大小
- 準確率仍然可以達到 87%左右
- FLOPs 維持在要求範圍內(<0.5G)

為什麼最終選擇 ShuffleNet V2:

1. 極低的模型大小:

- ShuffleNet V2 比 EfficientNet-B0 有更小的參數量

- 更容易滿足<20MB 的要求，並且有 10 倍的差距

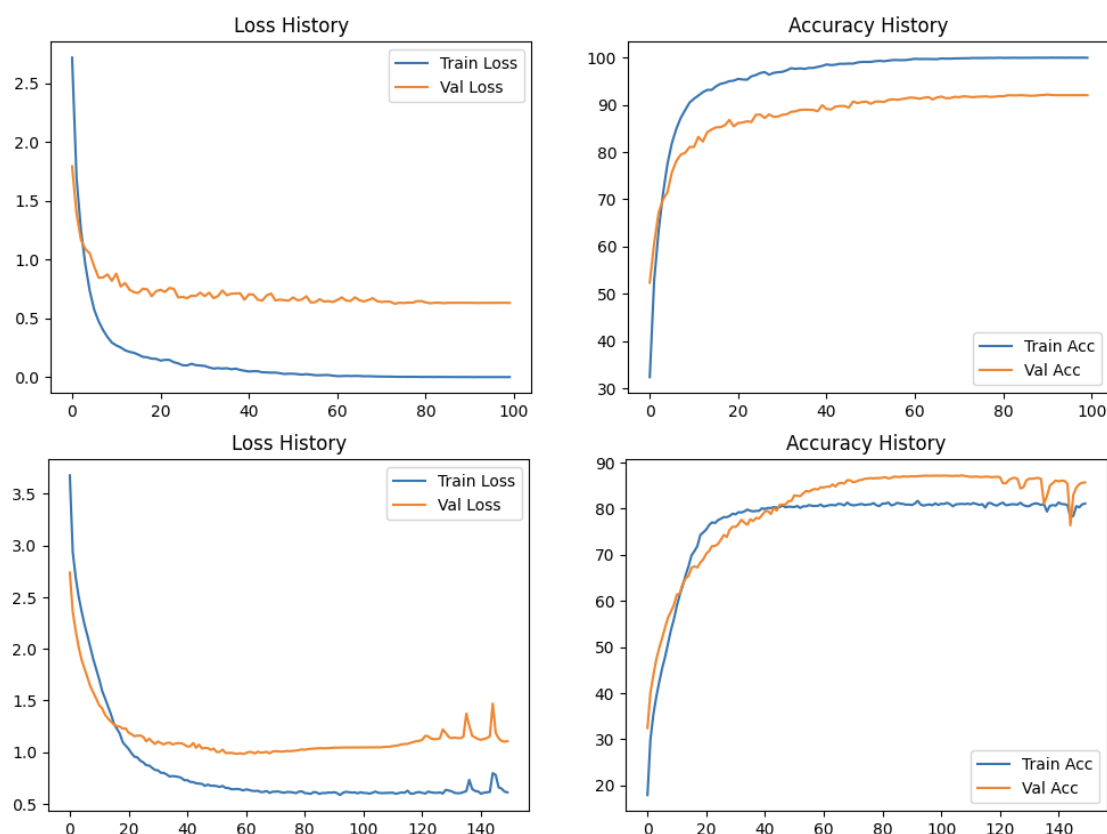
2. 計算效率:

- 專門優化的網絡架構
- 在保持準確率的同時大幅降低計算量

3. 實用性:

- 簡單的實現方式
- 穩定的訓練過程
- 良好的部署效率

通過以上設計, ShuffleNet V2 成功在模型大小、計算量和準確率之間取得了很好的平衡, 特別適合這個要求參數量小的任務。雖然準確率比 EfficientNet 略低, 但在 performance 的公式下是一個最好的選擇。



由於原本 train accuracy 達到 100%，為了避免 overfitting 我再額外增加 dropout 層，使得 validation accuracy 有再更上升。